

Teste de Software

Hypothesis - Property Testing

Universidade do Minho

2025/2026

1 Teste de Propriedades

O teste de código consiste em fornecer *inputs* e verificar os *outputs*. O mais comum é criar diversos testes para o mesmo caso, mas variar os dados fornecidos e resultados esperados. Contudo, esse método é laborioso e não especifica por completo o código.

Uma alternativa é definir propriedades sobre o código e deixar uma ferramenta gerar os *inputs*. Estas ferramentas, das quais o QuickCheck para a linguagem Haskell foi uma das primeiras, geram um conjunto definido de dados e passam-nos às propriedades. Existem, claro, outras ferramentas com capacidades semelhantes para outras linguagens de programação. Nesta ficha abordaremos o Hypothesis.

2 Exercícios

Os exercícios nesta ficha focam-se na definição de testes para código Python usando o Hypothesis¹. Para isto, será necessário garantir que o interpretador de Python está instalado na máquina, assim como as bibliotecas necessárias, correndo os comandos `pip install pytest` e `pip install hypothesis`.

Pode-se correr os testes unitários de um módulo Python com o comando `python -m pytest file.py`.

1. Tendo em conta as seguintes definições de `mulLR` (recursiva) e `mulLI` (iterativa):

```
def mulLR(list_nums):  
    if len(list_nums) == 0:  
        return 0  
    else:  
        return list_nums[0] * mulLR(list_nums[1:])
```

¹<https://hypothesis.readthedocs.io/en/latest/>

```
def mulLI(list_nums):
    res = 0
    for n in list_nums:
        res *= n
    return res
```

Defina as seguintes propriedades para as funções:

- (a) a multiplicação do inverso da lista (`<lista>[::-1]`) devolve o mesmo resultado;
- (b) o resultado para uma lista com um elemento é igual a esse elemento;
- (c) o resultado é igual ao resultado da função `prod` do módulo `math`.

Teste as propriedades definidas para verificar se as definições estão conforme a especificação.

2. Considere a seguinte definição da função `index`². Aqui usa-se o nome `myIndex` para evitar potenciais conflitos de nome com a função pré-definida.

```
def myIndex(input_list, value):
    if len(input_list) == 0:
        raise ValueError("Value not found in list")
    try:
        return 1 + myIndex(input_list[1:], value)
    except:
        if value == input_list[0]:
            return 0
        else:
            raise ValueError("Value not found in list")
```

que contém um erro na sua definição.

- (a) Defina propriedades que a função deve ter e teste-as.
 - i. É possível que as propriedades passem quando se esperava que falhassem. Verifique se é o caso aumentando o número de testes gerados e executados pelo Hypothesis manipulando a anotação `settings`.
 - (b) Corrija o código e teste novamente as propriedades.
3. Por vezes, comete-se o erro de se delegar o trabalho de geração à propriedade a ser testada em vez do gerador em si. Há ferramentas que permitem o uso condicional ou limitado de geradores, no entanto estas não devem ser abusadas. Pode-se limitar os valores testados através de uma pré-condição:

²<https://docs.python.org/3/tutorial/datastructures.html>

```
@given(integers())
def test_positive(x):
    assume(x > 0)
    assert (x * x * x) > 0
```

Ou especificar estratégias mais detalhadas de geração de valores:

```
@given(integers(min_value=1, max_value=1000))
def test_positive2(x):
    assert (x * x * x) > 0
```

Considere as seguintes funções para verificar se uma lista está ordenada e para inserir um elemento numa lista, respetivamente:

```
def isSorted(list_nums):
    return False not in [a <= b for (a,b) in zip(list_nums, list_nums[1:])]

def insert(elem, list_nums):
    i = 0
    r = []
    while (i < len(list_nums) and elem > list_nums[i]):
        r.append(list_nums[i])
        i += 1
    r.append(elem)
    while (i < len(list_nums)):
        r.append(list_nums[i])
        i += 1
    return r
```

- (a) Defina uma propriedade que, dado um inteiro e uma lista de inteiros, verifica que, caso a lista de inteiros esteja ordenada, então essa lista continua ordenada após a inserção do novo inteiro. Utilize a função **assume** nesta definição. Explique os resultados obtidos.
- (b) Acrescente à propriedade definida na alínea anterior a função **event**, que irá agrupar os resultados obtidos de acordo com a *String* que lhe fornecemos. Para se conseguir visualizar estes resultados, será necessário executar os testes com a flag **--hypothesis-show-statistics**. Sugestão: Agrupe os resultados de acordo com o tamanho da lista recebida como argumento da propriedade.
- (c) Defina um gerador de valores **sorted_intLists**, que produza listas de inteiros ordenados. Sugestão: Utilize o método **sorted** para ordenar listas de inteiros aleatórias. Investigue a estratégia **composite**³ do Hypothesis para a criação de novos geradores de valores.

³<https://hypothesis.readthedocs.io/en/latest/data.html#composite-strategies>

- (d) Defina uma propriedade que recebe um valor a inserir numa lista, usa o gerador `sorted_intLists` para gerar uma lista de valores, e verifica que se lista continua ordenada após inserção deste novo valor.

4. Considere a seguinte definição da classe `Aluno`:

```
class Aluno:
    def __init__(self, _nome, _idade, _curso, _notas):
        self.nome = _nome
        self.idade = _idade
        self.curso = _curso
        self.notas = _notas
```

Nesta classe, o `nome` e `curso` são strings, `idade` é um inteiro, e `notas` é um dicionário que mapeia nomes de UCs para listas de notas.

- (a) Defina as funções:
 - i. `__str__` para a impressão de um objeto da classe como string.
 - ii. `media` para calcular a média de todas as notas do aluno, sem distinção de UCs.
 - iii. `excelente` que devolverá `True` se e só se o aluno tiver todas as notas positivas e uma média maior ou igual a 15.
- (b) Defina geradores:
 - i. `gerador_nome` que gera um nome aleatório. Para isto, pode-se utilizar caracteres aleatórios (com `text(alphabet=string.ascii_letters)`, ou o gerador `sampled_from` que seleciona aleatoriamente um elemento de uma lista fornecida, ou o gerador `from_regex` que utiliza uma expressão regular para gerar valores).
 - ii. `gerador_idade` que gera uma idade válida.
 - iii. `gerador_curso` que gera um curso válido. Aqui vamos considerar apenas **LEI** e **LCC**.
 - iv. `gerador_notas` que gera um dicionário que associa UCs a listas de notas. Certifique-se que gera pelo menos 1 UC, que as listas de notas tem pelo menos 3 notas cada, e que as notas são válidas.
 - v. `gerador_aluno` que utiliza os geradores anteriores para gerar um `Aluno`.
- (c) Utilizando o gerador de alunos, defina propriedades:
 - i. `test_media` que certifica que a média de um aluno está entre 0 e 20.
 - ii. `test_media2` que certifica que o aluno não tem uma média superior à sua nota mais alta.
 - iii. `test_excelente` que verifica que, se um aluno for excelente, então a sua média será acima ou igual a 15.

- iv. outras propriedades que considere relevantes ou interessantes.
- (d) Defina uma função `imprimir_alunos` que, dado um número `n`, irá gerar um ficheiro `logs.txt` que tem `n` linhas, cada uma com um aluno gerado pelo gerador de alunos. Para gerar um valor com o gerador de alunos, poderá utilizar `gerador_aluno().example()`.